

PROJECT 2(P52)

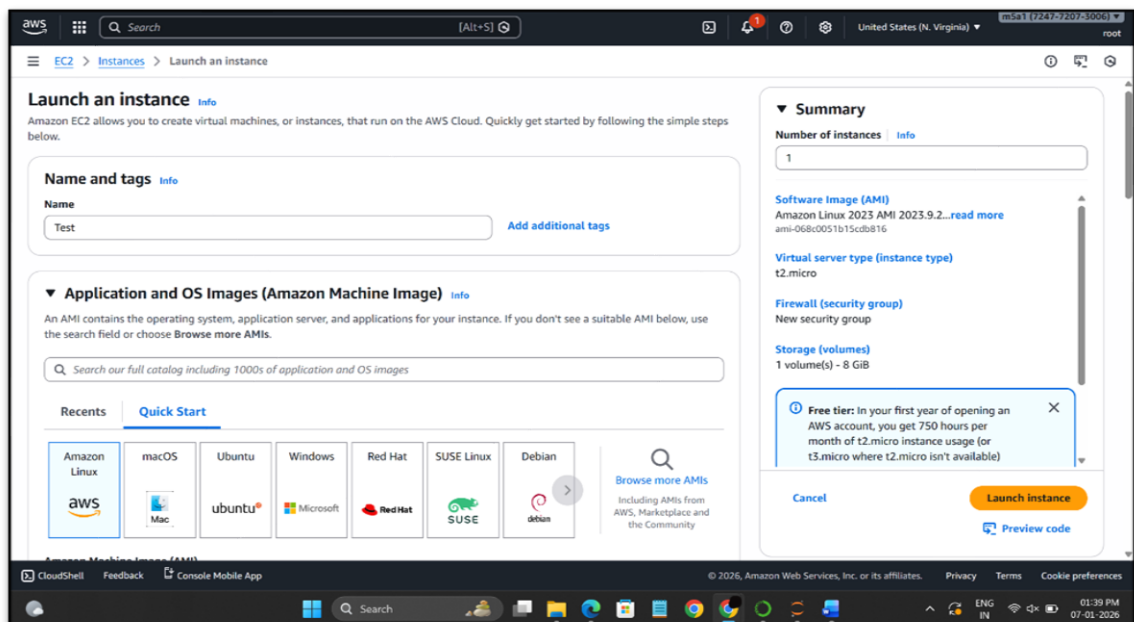
To Stop and Start Amazon EC2 Instances at regular time intervals using Lambda.

Requirements:

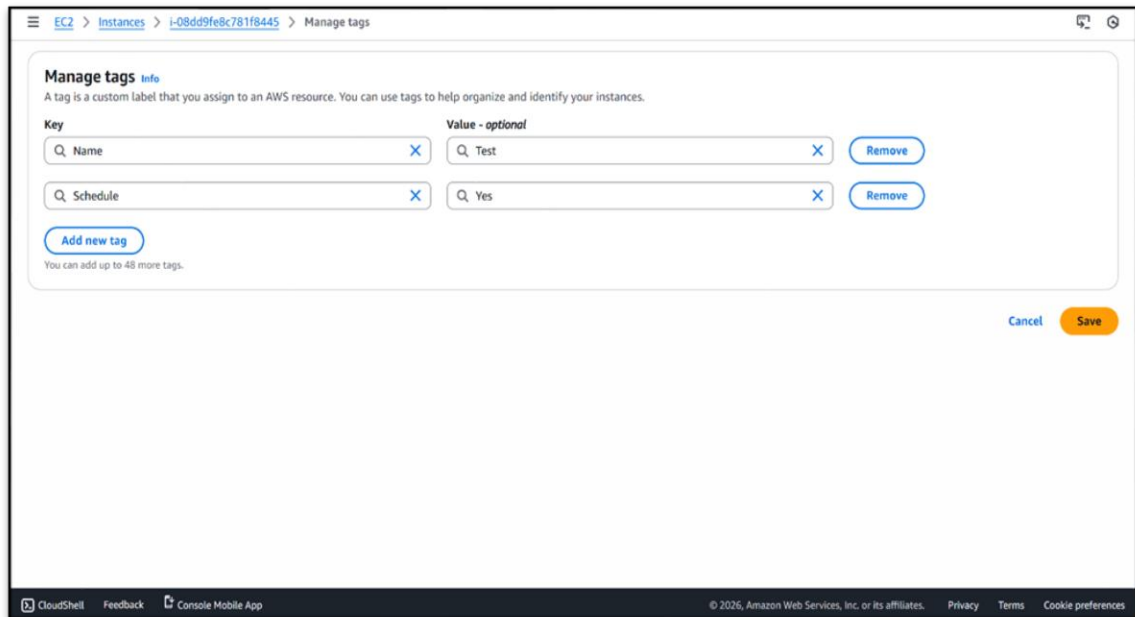
- Reduce EC2 Usage by scheduling start and stop automatically.
- Create IAM policy and execution role for Lambda function
- Create Lambda function to start and stop EC2 instances.
- Create EventBridge to rule Lambda function.

EC2:

- Open the AWS console→Navigate to EC2→Launch instance
→Name→Key pair→Allow SSH and HTTP in security group
→Launch instance.



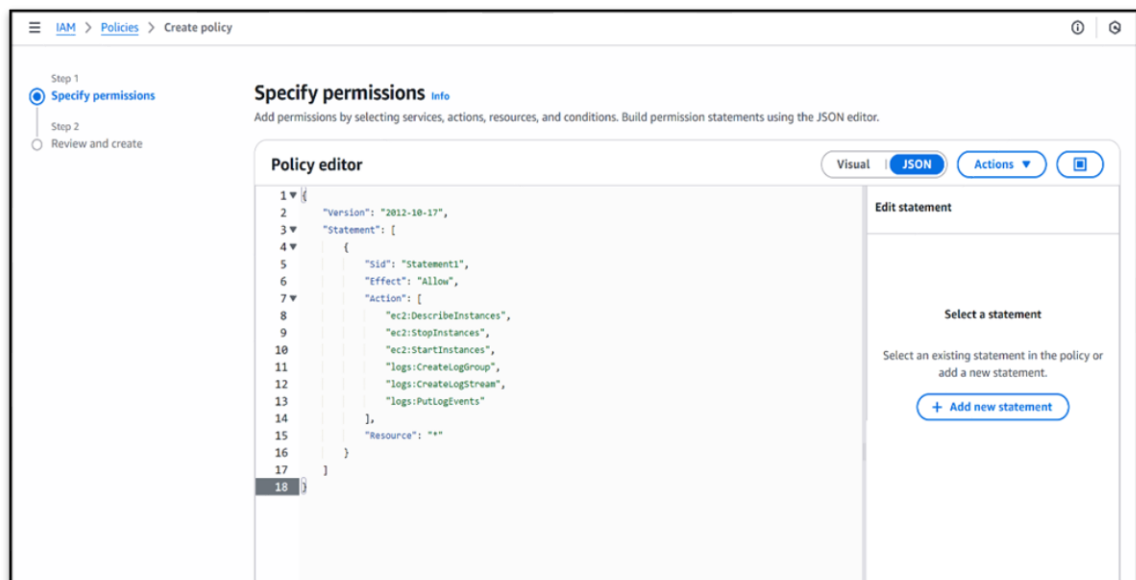
- After creation of instance select the instance→Navigate to tags→Manage tags→Add tags→Key='Schedule', Value='Yes'
→ Save. (This is done to help Lambda identify the instance)



IAM:

➤ Navigate to IAM policies → Create policy → JSON → Add the required permissions:

- EC2
- Logs



➤ Review and create: Policy name → "EC2LambdaPolicy" → Create policy.

Permissions defined in this policy [info](#) [Edit](#)

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it.

Search

Allow (2 of 461 services) [Show remaining 459 services](#)

Service	Access level	Resource	Request condition
CloudWatch Logs	Limited: Write	All resources	None
EC2	Limited: List, Write	All resources	None

Add tags - optional [info](#)

Tags are key-value pairs that you can add to AWS resources to help identify, organize, or search for resources.

No tags associated with the resource.

[Add new tag](#)

You can add up to 50 more tags.

[Cancel](#) [Previous](#) [Create policy](#)

- Navigate to Roles→Create Execution role→Select Lambda as service→Attach EC2 Lambda Policy→Next→Name:"Lambda EC2Role"→Create role.

Step 2: Add permissions [Edit](#)

Permissions policy summary

Policy name	Type	Attached as
EC2LambdaPolicy	Customer managed	Permissions policy

Step 3: Add tags

Add tags - optional [info](#)

Tags are key-value pairs that you can add to AWS resources to help identify, organize, or search for resources.

No tags associated with the resource.

[Add new tag](#)

You can add up to 50 more tags.

[Cancel](#) [Previous](#) [Create role](#)

Lambda:

- Navigate to Lambda Functions→Create function→Function name→Runtime(Python 3.12)→Use the existing Lambda EC2 Role→Create function.

Lambda > Functions > Create function

Choose the instruction set architecture you want for your function code.

☐ arm64
☒ x86_64

Permissions info
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

▼ **Change default execution role**

Execution role
Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).

☐ Create a new role with basic Lambda permissions
☒ Use an existing role
☐ Create a new role from AWS policy templates

Existing role
Choose an existing role that you've created to be used with this Lambda function. The role must have permission to upload logs to Amazon CloudWatch Logs.

LambdaEC2Role 

[View the LambdaEC2Role role](#) on the IAM console.

► **Additional configurations**
Use additional configurations to set up networking, security, and governance for your function. These settings help secure and customize your Lambda function deployment.

[Cancel](#) [Create function](#)

- Provide necessary code for the **Lambda Stop Function**→ Deploy.

Lambda > Functions > StopEC2Function

Code source info [Open in Visual Studio Code](#) [Upload from](#) 

EXPLORER

- STOPEC2FUNCTION
 - lambda_function.py

DEPLOY Current

[Deploy \(Ctrl+Shift+U\)](#)

[Test \(Ctrl+Shift+I\)](#)

TEST EVENTS (SELECTED: TESTSTOP)

- + Create new test event
- Private saved events
 - TestStop

```

1  import json
2
3  import boto3
4  def lambda_handler(event, context):
5      ec2 = boto3.client('ec2', region_name='us-east-1')
6      response = ec2.describe_instances(Filters=[{'Name': 'tag:Schedule', 'Values': ['Yes']}])
7      instance_ids = []
8      for reservation in response['Reservations']:
9          for instance in reservation['Instances']:
10             if instance['State']['Name'] == 'running':
11                 instance_ids.append(instance['InstanceId'])
12
13  if instance_ids:
14      ec2.stop_instances(InstanceIds=instance_ids)
15      return f'Stopped: {instance_ids}'
16  return 'No running instances tagged'
17

```

PROBLEMS OUTPUT CODE REFERENCE LOG TERMINAL

Tasks

- Configure the code by navigating to configuration→Enter the timeout seconds→Save.

Lambda > Functions > StopEC2Function > Edit basic settings

Ephemeral storage [Info](#)
 You can configure up to 10 GB of ephemeral storage (/tmp) for your function. [View pricing](#)
 MB
 Set ephemeral storage (/tmp) to between 512 MB and 10240 MB.

SnapStart [Info](#)
 Reduce startup time by having Lambda cache a snapshot of your function after the function has initialized. To evaluate whether your function code is resilient to snapshot operations, review the [SnapStart compatibility considerations](#). For Python and .NET runtimes, [view pricing](#).

 Supported runtimes: .NET 10 (C#/.NET Core), .NET 8 (C#/.NET Core), Java 11, Java 17, Java 21, Java 25, Python 3.12, Python 3.13, Python 3.14.

Timeout
 min sec

Execution role
 Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).
☒ Use an existing role
☐ Create a new role from AWS policy templates

Existing role
 Choose an existing role that you've created to be used with this Lambda function. The role must have permission to upload logs to Amazon CloudWatch Logs.

[View the LambdaEC2Role role](#) on the IAM console.

[Cancel](#) [Save](#)

- Test the Lambda function by navigating to Test→Proceed with empty JSON code→Save→Test.

Lambda > Functions > StopEC2Function > Edit basic settings

Ephemeral storage [Info](#)
 You can configure up to 10 GB of ephemeral storage (/tmp) for your function. [View pricing](#)
 MB
 Set ephemeral storage (/tmp) to between 512 MB and 10240 MB.

SnapStart [Info](#)
 Reduce startup time by having Lambda cache a snapshot of your function after the function has initialized. To evaluate whether your function code is resilient to snapshot operations, review the [SnapStart compatibility considerations](#). For Python and .NET runtimes, [view pricing](#).

 Supported runtimes: .NET 10 (C#/.NET Core), .NET 8 (C#/.NET Core), Java 11, Java 17, Java 21, Java 25, Python 3.12, Python 3.13, Python 3.14.

Timeout
 min sec

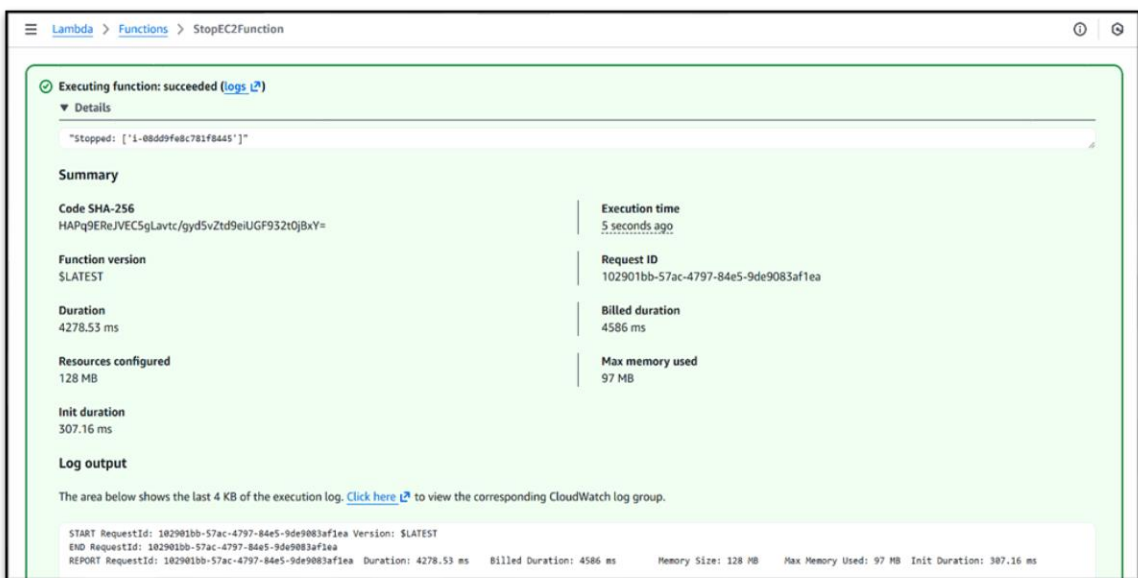
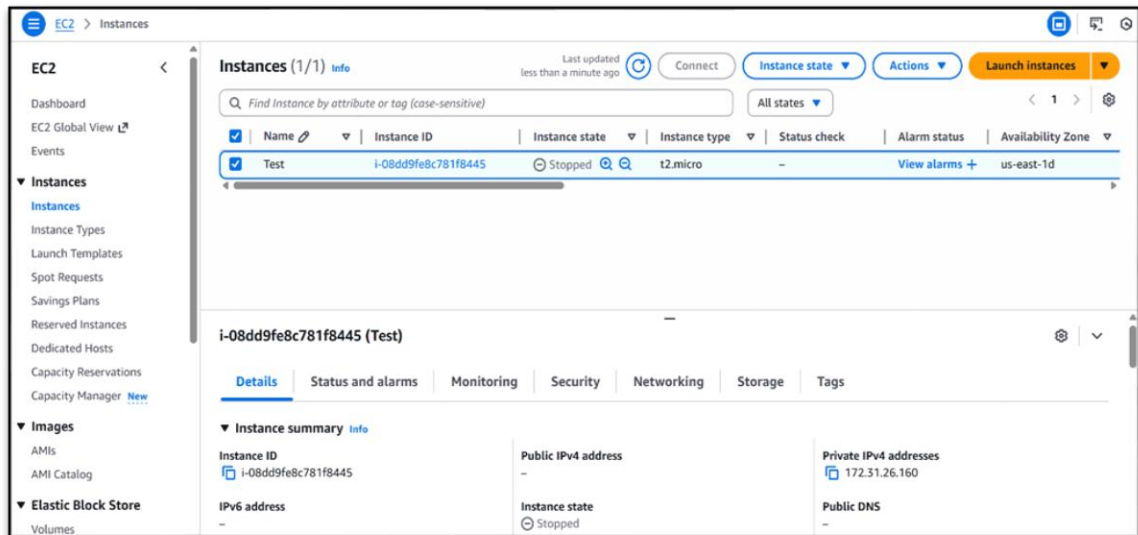
Execution role
 Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).
☒ Use an existing role
☐ Create a new role from AWS policy templates

Existing role
 Choose an existing role that you've created to be used with this Lambda function. The role must have permission to upload logs to Amazon CloudWatch Logs.

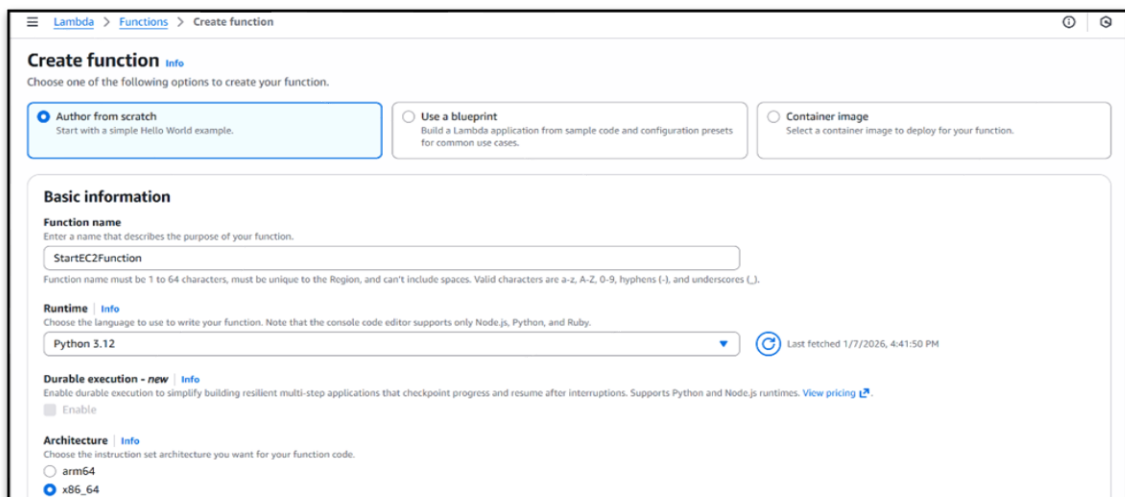
[View the LambdaEC2Role role](#) on the IAM console.

[Cancel](#) [Save](#)

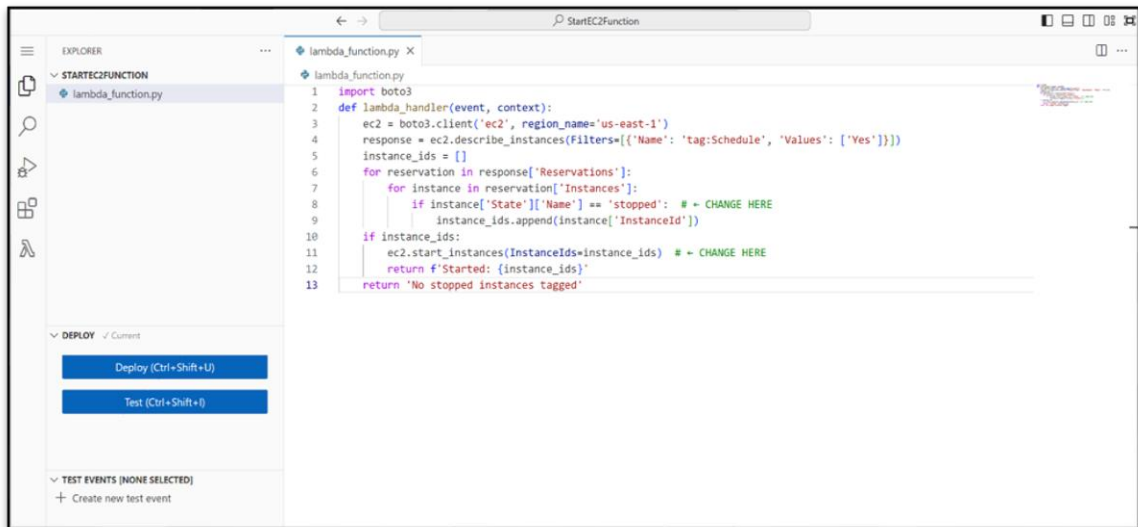
- You can find that the instance has been stopped and the stop function has been executed successfully.



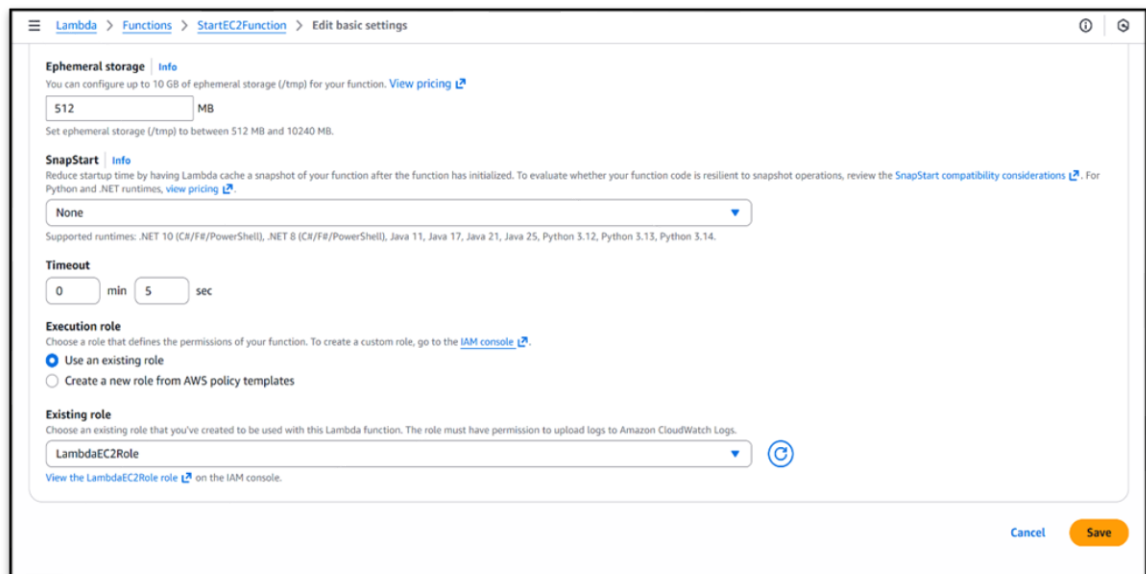
- Create Start Function by navigating to Lambda → Create function → Name: "StartEC2Function" → Provide the existing Lambda EC2 Role → Create function.



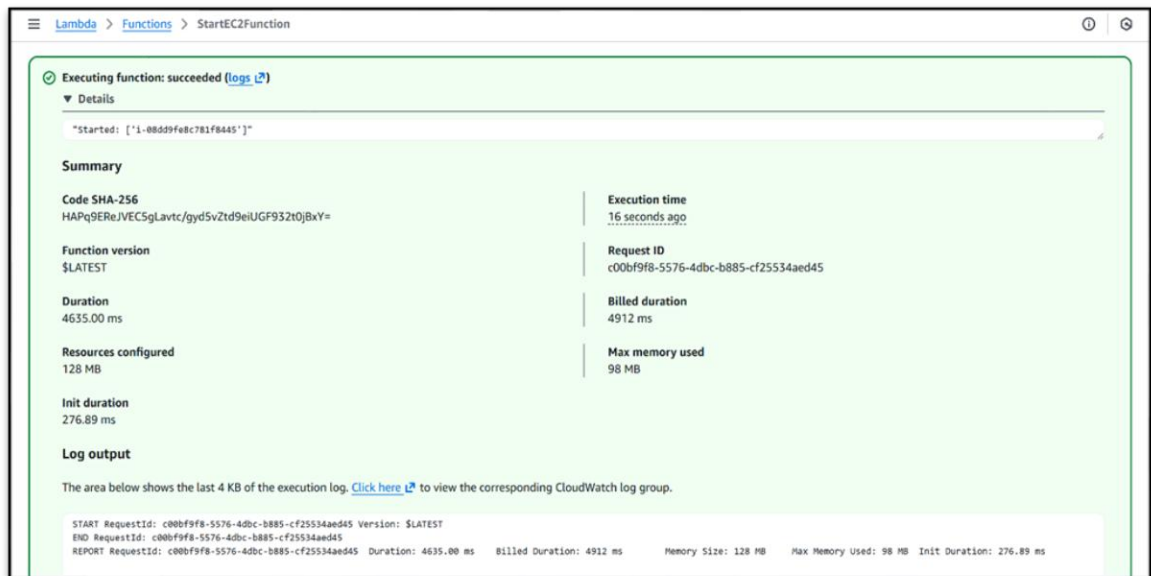
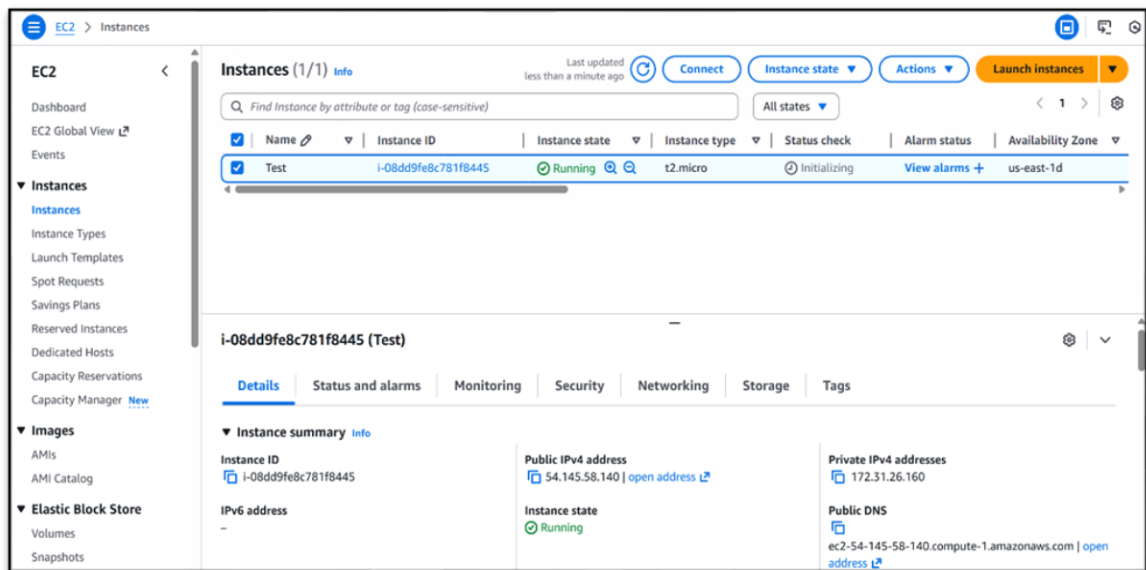
- Provide necessary code for the **Lambda Start Function**→
Deploy.



- Configure the code by navigating to configuration→Enter the
timeout seconds→Save.



- Test the Lambda function by navigating to Test→Proceed with
empty JSON code→Save→Test.
- You can find that the instance has been started and the start
function has been executed successfully.



EventBridge:

- Navigate to Amazon Eventbridge Console→Rules→Create rule
→Provide the name and description→Next.
- Define schedule by entering appropriate cron expression→Next.

The screenshot shows the 'Define schedule' step of the 'Create rule' wizard in Amazon EventBridge. The left sidebar contains navigation links for 'Amazon EventBridge', 'Dashboard', 'Developer resources', 'Buses', 'Pipes', 'Scheduler', and 'Integration'. The 'Rules' link is highlighted. The main content area is titled 'Define schedule' and includes a 'Schedule pattern' section with two options: 'A fine-grained schedule that runs at a specific time, such as 8:00 a.m. PST on the first Monday of every month.' (selected) and 'A schedule that runs at a regular rate, such as every 10 minutes.' Below this is a 'Cron expression' section with a field for the cron expression and a dropdown for the time zone (UTC). The 'Next 10 trigger date(s)' are listed below the cron expression.

Define schedule

Schedule pattern

Choose the schedule type that best meets your needs.

☒ A fine-grained schedule that runs at a specific time, such as 8:00 a.m. PST on the first Monday of every month.

☐ A schedule that runs at a regular rate, such as every 10 minutes.

Cron expression

Define the cron expression for the schedule

Minutes Hours Day of month Month Day of week Year

Next 10 trigger date(s)

Wed, 07 Jan 2026 12:00:00 UTC
Thu, 08 Jan 2026 12:00:00 UTC
Fri, 09 Jan 2026 12:00:00 UTC
Sat, 10 Jan 2026 12:00:00 UTC
Sun, 11 Jan 2026 12:00:00 UTC
Mon, 12 Jan 2026 12:00:00 UTC
Tue, 13 Jan 2026 12:00:00 UTC
Wed, 14 Jan 2026 12:00:00 UTC
Thu, 15 Jan 2026 12:00:00 UTC
Fri, 16 Jan 2026 12:00:00 UTC

Cancel Previous Next

- Select the target type as Lambda Function → Stop EC2 Function → Next → Review and Create.

The screenshot shows the 'Review and create' step of the 'Create rule' wizard in Amazon EventBridge. The left sidebar is the same as the previous screenshot. The main content area is titled 'Review and create' and includes a 'Step 1: Define rule detail' section with a table showing the rule name, status, event bus, and description. Below this is a 'Step 2: Build schedule' section with a table showing the cron expression, next trigger date, and next 10 trigger dates.

Review and create

Step 1: Define rule detail

Define rule detail	Status	Event bus
Rule name: TestStop6p	Enabled	default
Description: Stop at 6pmIST	Rule type: Scheduled rule	

Step 2: Build schedule

Event schedule
Cron expression: 0 12 * * ? *
Next 10 trigger date(s): UTC
Thu, 08 Jan 2026 12:00:00 UTC Fri, 09 Jan 2026 12:00:00 UTC Sat, 10 Jan 2026 12:00:00 UTC Sun, 11 Jan 2026 12:00:00 UTC Mon, 12 Jan 2026 12:00:00 UTC Tue, 13 Jan 2026 12:00:00 UTC

- Follow the same steps and create another rule for Start EC2 Function.

Amazon EventBridge > Rules > Create rule

Review and create

Step 1: Define rule detail

Define rule detail

Rule name: TestStart7pm
 Status: Enabled
 Description: Start at 7pmIST
 Rule type: Scheduled rule
 Event bus: default

Step 2: Build schedule

Event schedule

Cron expression: 0 13 * * *
 Next 10 trigger date(s): UTC
 Wed, 07 Jan 2026 13:00:00 UTC
 Thu, 08 Jan 2026 13:00:00 UTC
 Fri, 09 Jan 2026 13:00:00 UTC
 Sat, 10 Jan 2026 13:00:00 UTC
 Sun, 11 Jan 2026 13:00:00 UTC
 Mon, 12 Jan 2026 13:00:00 UTC

- Both the Start and Stop rule have been successfully created to schedule the EC2.

Amazon EventBridge > Rules

Rules

A rule watches for specific types of events. When a matching event occurs, the event is routed to the targets associated with the rule. A rule can be associated with one or more targets.

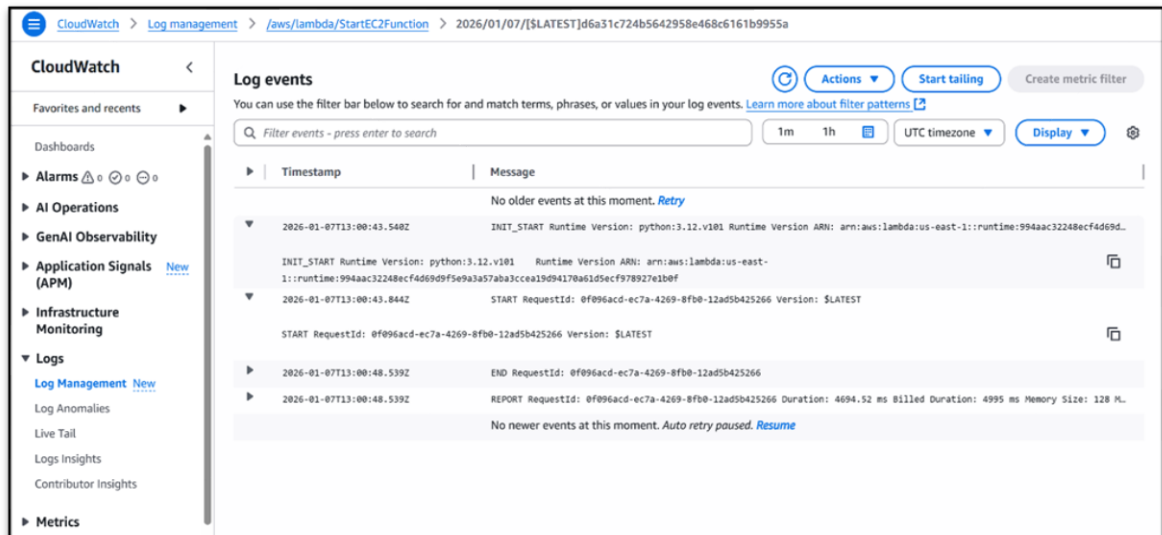
Select event bus

Event bus: default

Rules on default event bus (2)

Name	Status	Type	Event bus	ARN	Description
TestStart7pm	Enabled	Scheduled Standard	default	arn:aws:events:us-east-1:72477207:3006:rule/TestStart7pm	Start at 7pmIST
TestStop6pm	Enabled	Scheduled Standard	default	arn:aws:events:us-east-1:72477207:3006:rule/TestStop6pm	Stop at 6pmIST

- Through the log events we could find that instance have stopped and started at the specified time intervals.



- The EC2 instances have been scheduled to stop and start at the specified time using IAM policy, Lambda functions and Eventbridge.

Conclusion:

- This AWS EC2 auto stop/start project successfully implemented serverless automation using Lambda and EventBridge, achieving 75% cost savings by stopping instances during specified times.